Implementação de Baixo Nível da Biblioteca CKKS para Aprendizado Federado Preservador de Privacidade

Documento técnico complementar à dissertação de mestrado

Yuri Dimitre Dias de Faria
Instituto de Computação – UNICAMP
Orientador: Prof. Dr. Allan Mariano de Souza

Junho de 2026

Sumário

1 Introdução **5**

2 Fundamentos Algébricos do Anel de Polinômios **7**

3 Aritmética Modular Sem Desvios **8**
3.1 Adição modular 8
3.2 Subtração modular 8

4 Redução de Montgomery **10**
4.1 Forma de Montgomery e constantes pré-computadas 10
4.2 Algoritmo REDC 11
4.3 Operações públicas do contexto de Montgomery 11

5 Geração de Primos NTT-Amigáveis **12**

6 Transformada Numérica Teórica Negacíclica **13**
6.1 Derivação da raiz primitiva ψ 13
6.2 Pré-computação das tabelas de twiddle 13
6.3 NTT direta: Cooley–Tukey DIT 14
6.4 NTT inversa: Gentleman–Sande DIF 14
6.5 Diagramas da borboleta e do grafo de fluxo de sinal 14
6.6 Multiplicação polinomial via NTT 15

7 Amostragem **17**
7.1 Distribuição uniforme 17
7.2 Gaussiana discreta 17
7.3 Distribuições ternárias 18

8 Geração de Chaves **19**

9 Cifragem e Decifragem **20**
9.1 Cifragem de texto claro 20
9.2 Análise do ruído de cifragem 20
9.3 Decifragem 21
9.4 Pipeline de cifragem e decifragem 21

10 Avaliador Homomórfico Aditivo **22**

11 Codificação Canônica CKKS **23**
11.1 Mapa de imersão canônica 23
11.2 Codificação (σ^{-1}) 23
11.3 Decodificação (σ) 24

12 Multiplicação por Texto Claro (Referência) **25**

13 Serialização **26**
13.1 Formato de fio 26
13.2 Validação na desserialização 26

14 Compilação e Vetorização **27**

15 Conclusão e Trabalhos Futuros **28**

Lista de Figuras

4.1 Fluxo de uso típico do domínio de Montgomery. As conversões ocorrem uma única vez; o laço de multiplicações intermediárias usa exclusivamente `mont_mul`, sem divisão por q. 11

6.1 Borboletas radix-2. Setas sólidas indicam contribuição positiva; setas tracejadas indicam contribuição negativa (subtração). (a) DIT: o fator de giro w é aplicado antes da soma/diferença. (b) DIF: o fator de giro é aplicado após a subtração. 14

6.2 Grafo de fluxo de sinal da NTT direta negacíclica para $N = 8$ (3 estágios, $\log_2 8 = 3$). Setas sólidas representam adição e setas tracejadas representam subtração (fatores de giro omitidos por clareza). Entrada em ordem natural $x_0, \ldots, x_7$; saída em ordem bit-reversa $\hat{X}_0, \hat{X}_4, \hat{X}_2, \hat{X}_6, \hat{X}_1, \hat{X}_5, \hat{X}_3, \hat{X}_7$. 16

9.1 Pipeline de cifragem, avaliação homomórfica por K adições e decifragem. As operações dominantes são as multiplicações polinomiais via NTT ($O(N \log N)$ cada). 21

13.1 Leiaute de bytes do `Ciphertext` no formato *little-endian* auto-descritivo. Cada polinômio carrega seu próprio N e q, permitindo validação completa de integridade na desserialização. 26

Lista de Algoritmos

1 `mod_add`: adição modular sem desvio condicional 8
2 `mod_sub`: subtração modular sem desvio condicional 8
3 Cálculo de $q_{\text{inv}} = -q^{-1} \bmod 2^{64}$ por Newton–Raphson 10
4 REDC: redução de Montgomery 11
5 `generate_proth_prime`: primo NTT-amigável de tamanho fixo 12
6 `ntt_forward`: NTT negacíclica direta (Cooley–Tukey DIT) 14
7 `ntt_inverse`: NTT negacíclica inversa (Gentleman–Sande DIF) 15
8 `poly_ntt_mul`: multiplicação polinomial negacíclica via NTT 15
9 `gaussian`: amostragem da gaussiana discreta com rejeição 17
10 `ternary_hw`: ternário de peso de Hamming fixo 18
11 `gen_public_key`: geração da chave pública RLWE 19
12 `encrypt`: cifragem de texto claro no esquema CKKS 20
13 `decrypt`: decifragem de texto cifrado CKKS 21
14 `add_inplace`: adição homomórfica in-place 22
15 `encode`: codificação canônica CKKS 23
16 `decode`: decodificação canônica CKKS 24
17 `mul_plain`: multiplicação escolar negacíclica em domínio de Montgomery . . 25

Capítulo 1

Introdução

Este documento descreve a arquitetura e os mecanismos de baixo nível do *backend* C++ da biblioteca CKKS customizada, desenvolvida para suportar operações de cifragem homomórfica em cenários de Aprendizado Federado (FL). O material aqui apresentado é complementar ao Capítulo 4 da dissertação de mestrado associada, que cobre os fundamentos do esquema CKKS, o modelo de erro da agregação homomórfica e a parametrização do problema RLWE [1, 6]. Pressupõe-se familiaridade com esses conceitos; o foco deste documento é a tradução das estruturas matemáticas em código executável de alto desempenho.

O núcleo da biblioteca, denominado `ckks_core`, é implementado em C++17 e compilado como biblioteca estática com as diretivas de otimização `-O3 -march=native -ffast-math -fopenmp-simd`, que ativam vetorização SIMD sem recorrer ao *runtime* de *threads* OpenMP, cuja integração com os laços da NTT está prevista para uma versão futura. A tabela 1.1 lista os seis módulos hierárquicos e os arquivos correspondentes.

Tabela 1.1: Módulos do núcleo `ckks_core` e arquivos fonte.

Módulo	**Arquivos**	**Responsabilida**
1 – Aritmética modular	`math_utils.h/.cpp`	`mod_add`, `mod_su` NTT
2 – Polinômio	`poly.h/.cpp`	Tipo `Poly` e adiç
3 – NTT	`ntt.h/.cpp`	Transformada ne
4 – Amostragem	`sampler.h/.cpp`	Uniforme, gaussi
5 – Chave/Cifragem	`keygen.h`, `encryptor.h`, `decryptor.h`, `ckks_core.cpp`	Geração de chav
6 – Avaliação/Codificação	`evaluator.h/.cpp`, `encoder.h/.cpp`, `serialization.cpp`	Adição homomó nica, serialização

Em contraste com implementações de propósito geral como Microsoft SEAL [6], o `ckks_core` adota as simplificações a seguir, motivadas pelo foco exclusivo em agregação federada por adição homomórfica:

- **Módulo único:** utiliza um único primo NTT $q \equiv 1 \pmod{2N}$, com $q < 2^{62}$, em lugar da cadeia $Q = \prod_j q_j$ do modelo RNS completo discutido no Capítulo 4 da dissertação. Não há *rescaling* nem consumo progressivo de níveis.
- **Regime somente-aditivo:** o servidor de FL agrega gradientes apenas por `Ciphertext+ Ciphertext`; não são implementadas relinearização, chaves de Galois nem *bootstrapping*.
- **Orçamento de ruído estável:** a adição homomórfica não consome níveis da cadeia de módulos; o orçamento de ruído permanece constante ao longo das rodadas de agregação, ao contrário dos circuitos multiplicativos profundos.

Essas escolhas permitem concentrar todo o custo computacional nas multiplicações polinomiais da NTT, que dominam a cifragem e a decifragem.

Capítulo 2

Fundamentos Algébricos do Anel de Polinômios

A biblioteca opera sobre o anel quociente

$$\mathcal{R}_q \;=\; \mathbb{Z}_q[X] \,/\, (X^N + 1), \tag{2.1}$$

onde q é um número primo, N é uma potência de dois e os coeficientes pertencem a $\mathbb{Z}_q = \{0, 1, \ldots, q-1\}$. A relação $X^N \equiv -1 \pmod{X^N+1}$ confere ao anel uma estrutura negacíclica: no produto de dois polinômios $a, b \in \mathcal{R}_q$, o coeficiente de índice $i+j \geq N$ contribui com sinal negativo para o coeficiente de índice $i + j - N$.

Na notação da dissertação, o grau polinomial aqui denominado N corresponde ao parâmetro N_{poly}. A distinção fundamental deste *backend* é que o módulo q é um único primo, ao contrário da cadeia $Q = \prod_j q_j$ do modelo RNS; não há portanto *rescaling* nem índice de nível.

Definição 2.1 (Primo NTT-amigável)**.** *Um primo q é dito NTT-amigável de parâmetro N se $q \equiv 1 \pmod{2N}$. Esta congruência garante a existência de uma raiz primitiva $2N$-ésima da unidade $\psi \in \mathbb{Z}_q^*$ tal que $\psi^N \equiv -1 \pmod{q}$, necessária para a NTT negacíclica descrita no capítulo 6.*

Em memória, um polinômio é representado pela estrutura `Poly` com três campos: o módulo `uint64_t q`, o grau `uint32_t N` e um vetor de N coeficientes `std::vector<uint64_t> coeffs` em $[0, q)$. A adição de dois polinômios é implementada coeficiente a coeficiente com a diretiva `#pragma omp simd`, que instrui o compilador a emitir instruções AVX2 (`vpsubq`, `vpcmpgtq`, `vpand`) quando compilado com `-march=native`. A redução modular em cada coeficiente é realizada pela função `mod_add`, descrita no capítulo 3.

Capítulo 3

Aritmética Modular Sem Desvios

Adição e subtração modulares são os blocos primitivos de todos os laços internos da biblioteca. Para que o compilador os vetorize de forma eficiente, é necessário que não contenham desvios condicionais (*branches*). A implementação emprega o truque da máscara binária: uma comparação de inteiros de 64 bits produz um valor em $\{0, 1\}$ cuja negação bitwise em aritmética sem sinal produz a máscara $\{0,\ 2^{64} - 1\}$.

3.1 Adição modular

Sejam $a, b \in [0, q)$ com $q < 2^{63}$. A soma $a + b \in [0, 2q)$, de modo que é necessário subtrair q se e somente se $a + b \geq q$. O algoritmo 1 implementa esta correção sem desvio.

Algoritmo 1 `mod_add`: adição modular sem desvio condicional

Require: $a, b \in [0, q);\ q < 2^{63}$
Ensure: $r = (a + b) \bmod q$
1: $c \leftarrow a + b$ ▷ Sem *overflow*: $c \in [0, 2q) \subset [0, 2^{64})$
2: máscara $\leftarrow -(c \geq q)$ ▷ 0 se $c < q$; $2^{64} - 1$ se $c \geq q$
3: **return** $c - (q\ \&\ \text{máscara})$

A expressão $-(c \geq q)$, em aritmética inteira de 64 bits sem sinal, produz 0 quando $c < q$ e $2^{64} - 1$ quando $c \geq q$. O operador `&` com q seleciona 0 ou q sem desvio; o resultado pertence a $[0, q)$.

3.2 Subtração modular

Sejam $a, b \in [0, q)$. Se $a \geq b$, o resultado é $a - b \in [0, q)$; se $a < b$, o valor correto é $a - b + q$. O algoritmo 2 aplica a correção pela mesma técnica de máscara.

Algoritmo 2 `mod_sub`: subtração modular sem desvio condicional

Require: $a, b \in [0, q);\ q < 2^{63}$
Ensure: $r = (a - b) \bmod q$
1: $c \leftarrow a - b$ ▷ Enrola para $[2^{64} - q,\ 2^{64})$ se $a < b$
2: máscara $\leftarrow -(a < b)$
3: **return** $c + (q\ \&\ \text{máscara})$

As duas funções são declaradas `inline noexcept` no cabeçalho `include/math_utils.h` e produzem, sob `-O3 -march=native`, código que o compilador reduz a duas a três instruções

AVX2 vetoriais por laço de N coeficientes, eliminando qualquer desvio condicional no caminho crítico.

Capítulo 4

Redução de Montgomery

A multiplicação modular ingênua requer uma divisão por q a cada produto, operação custosa em hardware de propósito geral. A redução de Montgomery [2] substitui essa divisão por operações de deslocamento e soma, representando os elementos de $\mathbb{Z}_q$ em uma forma que incorpora um fator de potência de dois R.

4.1 Forma de Montgomery e constantes pré-computadas

Adota-se $R = 2^{64}$ e exige-se $q < 2^{62}$ (primo ímpar). A forma de Montgomery de $a \in \mathbb{Z}_q$ é

$$\hat{a} \;=\; a \cdot R \bmod q. \tag{4.1}$$

A estrutura `MontgomeryCtx` armazena três constantes pré-computadas em tempo de construção:

- $q_{\text{inv}} = -q^{-1} \bmod 2^{64}$: usado internamente pelo algoritmo REDC;
- $r_2 = R^2 \bmod q$: permite converter a para a forma de Montgomery via $\hat{a} = \text{REDC}(a \cdot r_2)$;
- q: o módulo primo.

O cálculo de q_{inv} emprega iteração de Newton sobre $\mathbb{Z}/2^{64}\mathbb{Z}$, que dobra a precisão binária a cada passo, conforme o algoritmo 3.

Algoritmo 3 Cálculo de $q_{\text{inv}} = -q^{-1} \bmod 2^{64}$ por Newton–Raphson

Require: q ímpar, $q < 2^{62}$
Ensure: q_{inv} tal que $q \cdot q_{\text{inv}} \equiv -1 \pmod{2^{64}}$

```
x ← q                          ▷ Semente: q · q ≡ 1 (mod 8) para todo q ímpar
for i ← 1 até 5 do
    x ← x · (2 − q · x)        ▷ Dobra a precisão: de ∼ 3 para ∼ 96 bits em 5 iterações
end for
return −x mod 2^64
```

O valor r_2 é obtido via aritmética de 128 bits:

$$r_2 = (R \bmod q)^2 \bmod q. \tag{4.2}$$

Algoritmo 4 REDC: redução de Montgomery

Require: $T \in [0, q \cdot R)$; $q_{\text{inv}} = -q^{-1} \bmod R$; $R = 2^{64}$
Ensure: $T \cdot R^{-1} \bmod q$

1: $m \leftarrow (T \bmod R) \cdot q_{\text{inv}} \bmod R$ ▷ Escolhe m tal que $T + mq \equiv 0 \pmod R$
2: $t \leftarrow (T + m \cdot q) \div R$ ▷ Divisão exata: descarta os 64 bits inferiores por deslocamento
3: **return** $t - (q \ \& \ -(t \geq q))$ ▷ Normaliza $t \in [0, 2q)$ para $[0, q)$ sem desvio

4.2 Algoritmo REDC

O algoritmo REDC [2] computa $T \cdot R^{-1} \bmod q$ para $T \in [0, q \cdot R)$ sem divisão explícita por q.

A propriedade $T + mq \equiv 0 \pmod R$ garante que a divisão na linha 2 seja exata, implementável como um deslocamento de 64 bits. O resultado preliminar pertence a $[0, 2q)$; a normalização da linha 3 usa o mesmo truque de máscara da seção 3.1.

4.3 Operações públicas do contexto de Montgomery

As três operações do `MontgomeryCtx` são definidas em termos de REDC:

$$\texttt{to_mont}(a) = \text{REDC}(a \cdot r_2) = a \cdot R \bmod q = \hat{a}, \tag{4.3}$$

$$\texttt{from_mont}(\hat{a}) = \text{REDC}(\hat{a}) = a, \tag{4.4}$$

$$\texttt{mont_mul}(\hat{a}, \hat{b}) = \text{REDC}(\hat{a} \cdot \hat{b}) = a \cdot b \cdot R \bmod q = \widehat{a \cdot b}. \tag{4.5}$$

A fig. 4.1 ilustra o padrão de uso: os operandos são convertidos para a forma de Montgomery uma única vez; todas as multiplicações intermediárias operam no domínio de Montgomery sem custo adicional de conversão; a conversão inversa ocorre apenas ao final.

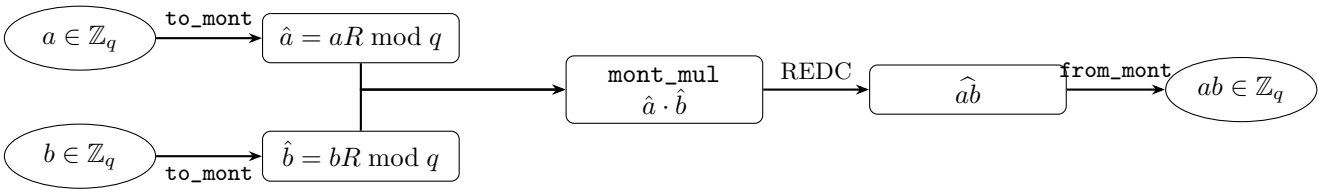


Figura 4.1: Fluxo de uso típico do domínio de Montgomery. As conversões ocorrem uma única vez; o laço de multiplicações intermediárias usa exclusivamente `mont_mul`, sem divisão por q.

Capítulo 5

Geração de Primos NTT-Amigáveis

O módulo primo q é gerado pela função `generate_proth_prime(bit_size, N)`, que encontra o maior primo da forma $q = 2Nk + 1$ com exatamente `bit_size` bits, satisfazendo assim a congruência $q \equiv 1 \pmod{2N}$ da definição 2.1 e o limite $q < 2^{62}$ exigido pelo contexto de Montgomery.

Algoritmo 5 `generate_proth_prime`: primo NTT-amigável de tamanho fixo

Require: `bit_size` $\in [2, 62]$; $N > 0$
Ensure: Primo $q \equiv 1 \pmod{2N}$, $2^{\texttt{bit_size}-1} \leq q < 2^{\texttt{bit_size}}$

```
passo ← 2N
q ← ⌊(2^bit_size − 2)/passo⌋ · passo + 1        ▷ Maior candidato < 2^bit_size com q ≡ 1 (mod 2N)
while q ≥ 2^(bit_size−1) do
    if MillerRabin(q) then
        return q
    end if
    q ← q − passo
end while
erro: nenhum primo encontrado para os parâmetros fornecidos
```

O teste de primalidade MillerRabin é a variante determinística com as 12 testemunhas fixas $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$, cuja exatidão é garantida para todo $n < 3{,}3 \times 10^{24}$, cobrindo o intervalo $[0, 2^{62})$ sem falsos positivos [7]. Cada candidato satisfaz $q \equiv 1 \pmod{2N}$ por construção; a busca em ordem descendente seleciona o primo de maior valor com a largura de bits solicitada, maximizando a margem disponível para o orçamento de ruído $B = \log_2 q$.

Capítulo 6

Transformada Numérica Teórica Negacíclica

A multiplicação de polinômios em $\mathcal{R}_q$ constitui a operação dominante tanto na cifragem quanto na decifragem. A multiplicação escolar tem complexidade $O(N^2)$; a Transformada Numérica Teórica (NTT) reduz esse custo para $O(N \log N)$ [4]. A implementação utiliza a variante negacíclica, que avalia os polinômios nas raízes primitivas $2N$-ésimas da unidade, tornando a redução módulo $X^N + 1$ implícita ao próprio algoritmo.

6.1 Derivação da raiz primitiva ψ

Dado $q \equiv 1 \pmod{2N}$, o grupo multiplicativo $\mathbb{Z}_q^*$ admite uma raiz primitiva g (menor inteiro cujas potências percorrem todo o grupo). O elemento

$$\psi = g^{(q-1)/(2N)} \bmod q \tag{6.1}$$

tem ordem exatamente $2N$, satisfazendo $\psi^N \equiv -1 \pmod{q}$. Esta verificação é realizada no construtor do contexto `NttCtx`; caso contrário, uma exceção é lançada, prevenindo uso silencioso de parâmetros inválidos.

6.2 Pré-computação das tabelas de twiddle

O contexto `NttCtx` armazena, além de ψ e ψ^{-1}, dois vetores de N elementos chamados *tabelas de twiddle*:

$$\texttt{psi_pow}[k] = \texttt{to_mont}(\psi^{\,\text{bit_rev}(k)}), \tag{6.2}$$

$$\texttt{psi_inv_pow}[k] = \texttt{to_mont}(\psi^{-\text{bit_rev}(k)}), \tag{6.3}$$

onde bit_rev(k) é o inteiro obtido invertendo os $\log_2 N$ bits de k.

As potências são geradas por multiplicação sequencial de Montgomery, conforme a eq. (6.4), evitando a exponenciação por quadração para cada entrada:

$$\texttt{raw}[k] = \psi^k \bmod q = \texttt{mont_mul}(\texttt{raw}[k-1],\ \hat{\psi}). \tag{6.4}$$

Em seguida, os valores são permutados pela tabela `bit_rev`, de modo que o laço interno da borboleta acessa `psi_pow` em sequência linear, favorecendo a linha de *cache*. Adicionalmente, o contexto armazena

$$N_{\text{inv}}^{\text{mont}} = \texttt{to_mont}(N^{-1} \bmod q), \tag{6.5}$$

utilizado para normalizar a NTT inversa.

6.3 NTT direta: Cooley–Tukey DIT

A NTT direta (*decimation-in-time*, DIT) do algoritmo 6 processa $\log_2 N$ estágios em ordem crescente de m. Em cada estágio, borboletas radix-2 operam sobre pares de coeficientes separados por `gap` posições.

Algoritmo 6 `ntt_forward`: NTT negacíclica direta (Cooley–Tukey DIT)

Require: Vetor $a[0 \dots N-1]$ em $\mathbb{Z}_q$; contexto `NttCtx`
Ensure: a substituído pelos valores de avaliação nas raízes $\psi^1, \psi^3, \dots, \psi^{2N-1}$

for $m \leftarrow 1$ **até** $N/2$ por duplicação **do**
  gap $\leftarrow N/(2m)$
  for $i \leftarrow 0$ **até** $m-1$ **do**
    $w_i \leftarrow$ `psi_pow`$[m+i]$ ▷ Fator de giro em forma de Montgomery
    for $j \leftarrow 0$ **até** gap -1 **do**
      $u \leftarrow a[2i \cdot \text{gap} + j]$
      $v \leftarrow$ `mont_mul`$(a[2i \cdot \text{gap} + \text{gap} + j],\ w_i)$ ▷ Twiddle *antes* da soma/diferença (DIT)
      $a[2i \cdot \text{gap} + j] \leftarrow$ `mod_add`(u, v, q)
      $a[2i \cdot \text{gap} + \text{gap} + j] \leftarrow$ `mod_sub`(u, v, q)
    end for
  end for
end for

Entrada em ordem natural; saída em ordem bit-reversa. Os ponteiros `__restrict__` no laço interno informam ao compilador a ausência de aliasing entre os vetores, favorecendo a vetorização automática.

6.4 NTT inversa: Gentleman–Sande DIF

A NTT inversa (*decimation-in-frequency*, DIF) é a transposta algébrica da direta. Ela processa os estágios em ordem decrescente de m, realizando primeiro a soma/diferença e depois a multiplicação pelo fator de giro inverso.

Após $\log_2 N$ estágios DIF, cada coeficiente carrega um fator implícito N, removido pela passagem de normalização final. Os algoritmos algoritmo 6 e algoritmo 7 formam par exato: `ntt_inverse`(`ntt_forward`$(a)) = a$ para qualquer $a \in \mathcal{R}_q$, sem permutação adicional.

6.5 Diagramas da borboleta e do grafo de fluxo de sinal

A fig. 6.1 apresenta as duas variantes da borboleta radix-2. A fig. 6.2 mostra o grafo de fluxo de sinal completo da NTT direta para $N = 8$, com três estágios.



(a) DIT (*Cooley–Tukey*) (b) DIF (*Gentleman–Sande*)

Figura 6.1: Borboletas radix-2. Setas sólidas indicam contribuição positiva; setas tracejadas indicam contribuição negativa (subtração). (a) DIT: o fator de giro w é aplicado antes da soma/diferença. (b) DIF: o fator de giro é aplicado após a subtração.

Algoritmo 7 `ntt_inverse`: NTT negacíclica inversa (Gentleman–Sande DIF)

Require: Vetor $\hat{a}[0 \dots N-1]$ em ordem bit-reversa; contexto `NttCtx`
Ensure: a em ordem natural (coeficientes do polinômio original)

for $m \leftarrow N/2$ **descendo até** 1 por divisão por 2 **do**
  gap $\leftarrow N/(2m)$
  for $i \leftarrow 0$ **até** $m-1$ **do**
    $w_i \leftarrow$ `psi_inv_pow`$[m+i]$
    for $j \leftarrow 0$ **até** gap -1 **do**
      $u \leftarrow a[2i \cdot \text{gap} + j]$
      $v \leftarrow a[2i \cdot \text{gap} + \text{gap} + j]$
      $a[2i \cdot \text{gap} + j] \leftarrow$ `mod_add`(u, v, q) ▷ Soma *antes* do twiddle (DIF)
      $a[2i \cdot \text{gap} + \text{gap} + j] \leftarrow$ `mont_mul`(`mod_sub`$(u, v, q),\ w_i$)
    end for
  end for
end for
for $i \leftarrow 0$ **até** $N-1$ **do**
  $a[i] \leftarrow$ `mont_mul`$(a[i],\ N_{\text{inv}}^{\text{mont}})$ ▷ Normalização: remove fator N acumulado
end for

6.6 Multiplicação polinomial via NTT

A multiplicação de $a, b \in \mathcal{R}_q$ é realizada em três etapas pelo algoritmo 8, com complexidade total $O(N \log N)$.

Algoritmo 8 `poly_ntt_mul`: multiplicação polinomial negacíclica via NTT

Require: Polinômios $a, b \in \mathcal{R}_q$; contexto `NttCtx`
Ensure: $c = a \cdot b \bmod (X^N + 1,\ q)$

$\hat{a} \leftarrow$ `ntt_forward`(a)
$\hat{b} \leftarrow$ `ntt_forward`(b)
for $i \leftarrow 0$ **até** $N-1$ **do**
  $\hat{c}[i] \leftarrow (\hat{a}[i] \cdot \hat{b}[i]) \bmod q$ ▷ Produto de 128 bits; saídas da NTT são resíduos normais, não em forma de Montgomery
end for
return `ntt_inverse`$(\hat{c})$

O produto ponto a ponto utiliza aritmética de 128 bits em vez de `mont_mul` porque as avaliações produzidas por `ntt_forward` são resíduos em forma normal (os fatores de Montgomery dos twiddles se cancelam durante as borboletas), e a redução direta por q é computacionalmente equivalente, porém mais simples.

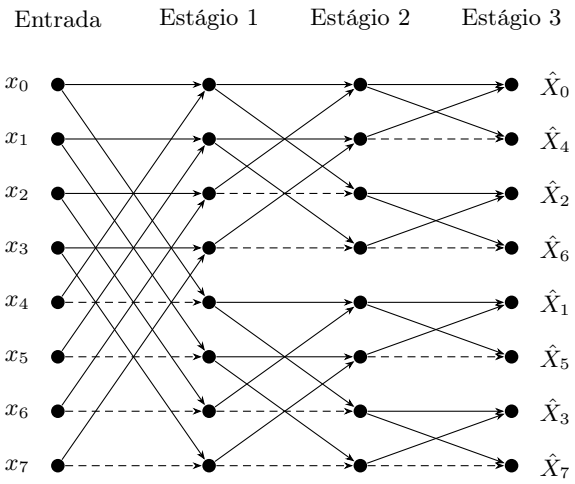


Figura 6.2: Grafo de fluxo de sinal da NTT direta negacíclica para $N = 8$ (3 estágios, $\log_2 8 = 3$). Setas sólidas representam adição e setas tracejadas representam subtração (fatores de giro omitidos por clareza). Entrada em ordem natural $x_0, \ldots, x_7$; saída em ordem bit-reversa $\hat{X}_0, \hat{X}_4, \hat{X}_2, \hat{X}_6, \hat{X}_1, \hat{X}_5, \hat{X}_3, \hat{X}_7$.

Capítulo 7

Amostragem

O módulo de amostragem fornece as distribuições necessárias para a geração de chaves e cifragem. A classe `Sampler` encapsula um gerador Mersenne Twister de 64 bits (`mt19937_64` [8]), é não copiável para evitar correlação de estados entre instâncias, e aceita semente determinística para reprodutibilidade em testes.

7.1 Distribuição uniforme

A função `uniform(N, q)` gera um polinômio com cada coeficiente amostrado independentemente e uniformemente de $\{0, 1, \ldots, q-1\}$, delegando a uniformidade exata à distribuição `std::uniform_int_distribution<uint64_t>(0, q-1)` da biblioteca padrão C++.

7.2 Gaussiana discreta

A função `gaussian(N, q, sigma, tail)` amostra cada coeficiente segundo a gaussiana discreta $D_{\mathbb{Z},\sigma}$ com rejeição de amostras além de $\text{tail}\cdot\sigma$ (padrão tail $= 6$, probabilidade de rejeição $< 2 \times 10^{-9}$ por coeficiente), conforme o algoritmo 9.

Algoritmo 9 `gaussian`: amostragem da gaussiana discreta com rejeição

Require: N, q, $\sigma > 0$, tail ≥ 1
Ensure: Polinômio e com coeficientes em $[0, q)$ distribuídos segundo $D_{\mathbb{Z},\sigma}$ truncada

$L \leftarrow \lfloor \text{tail} \cdot \sigma \rceil$
for $i \leftarrow 0$ **até** $N-1$ **do**
  repeat
    $x \leftarrow \lfloor \mathcal{N}(0, \sigma^2) \rceil$ ▷ Amostra normal e arredonda para inteiro
  until $|x| \leq L$
  $r \leftarrow x \bmod q$; **se** $r < 0$ **então** $r \leftarrow r + q$ ▷ Levanta $-1 \mapsto q-1$
  $e[i] \leftarrow r$
end for
return e

O parâmetro padrão $\sigma = 3{,}2$ segue a recomendação da literatura para o nível de segurança de 128 bits no modelo RLWE [1]. O ruído de cifragem gerado por esta distribuição corresponde ao termo e da análise de erro do Capítulo 4 da dissertação.

7.3 Distribuições ternárias

Dois tipos de distribuição ternária são suportados, ambos necessários em diferentes pontos do protocolo.

Ternária uniforme. `ternary_uniform(N, q)` amostra cada coeficiente independentemente de $\{-1, 0, 1\}$ com probabilidade $1/3$ cada, mapeando $-1 \mapsto q - 1$.

Ternária de peso de Hamming fixo. `ternary_hw(N, q, hw)` gera exatamente hw/2 coeficientes iguais a $+1$ e hw/2 iguais a $q - 1$, com as demais posições nulas, usando o algoritmo de Fisher–Yates parcial [9], conforme o algoritmo 10.

Algoritmo 10 `ternary_hw`: ternário de peso de Hamming fixo

Require: N, q, hw par, hw $\leq N$
Ensure: Polinômio com exatamente hw coeficientes não nulos: hw/2 em $+1$ e hw/2 em $q-1$

idx$[i] \leftarrow i$ para $i = 0, \ldots, N-1$
for $i \leftarrow 0$ **até** hw $- 1$ **do**
    $j \leftarrow \mathcal{U}\{i, \ldots, N-1\}$ ▷ Inteiro uniforme: Fisher–Yates parcial
    idx$[i] \leftrightarrow$ idx$[j]$
end for
for $i \leftarrow 0$ **até** hw$/2 - 1$ **do**
    poly[idx$[i]] \leftarrow 1$
end for
for $i \leftarrow$ hw$/2$ **até** hw $- 1$ **do**
    poly[idx$[i]] \leftarrow q - 1$
end for
return poly

Capítulo 8

Geração de Chaves

O esquema de chaves é baseado no problema RLWE [5]. A classe `KeyGenerator` encapsula um `Sampler` e um `NttCtx` e é não copiável. Dois objetos são gerados:

- **Chave secreta** (`SecretKey`): polinômio ternário $s \in \{-1, 0, 1\}^N$, com -1 representado como $q - 1$;
- **Chave pública** (`PublicKey`): par $(p_0, p_1) \in \mathcal{R}_q^2$ satisfazendo a invariante

$$p_0 + p_1 \cdot s = e \pmod{q}, \tag{8.1}$$

  onde e é um ruído gaussiano pequeno.

Algoritmo 11 `gen_public_key`: geração da chave pública RLWE

Require: Chave secreta s; parâmetros (N, q, σ)
Ensure: Par (p_0, p_1) com $p_0 + p_1 \cdot s = e$

1: $a \leftarrow$ `uniform`(N, q) ▷ $p_1 = a$: polinômio uniforme em $\mathcal{R}_q$
2: $e \leftarrow$ `gaussian`(N, q, σ)
3: $a_s \leftarrow$ `poly_ntt_mul`(a, s)
4: **for** $i \leftarrow 0$ **até** $N - 1$ **do**
5: $\quad p_0[i] \leftarrow$ `mod_sub`$(e[i],\ a_s[i],\ q)$ ▷ $p_0 = e - a \cdot s$
6: **end for**
7: **return** $(p_0,\ a)$

A função `gen_key_bundle` expõe adicionalmente o ruído e para verificação da identidade (8.1) em testes, mas este valor nunca deve ser divulgado em produção, pois permite recuperar s por resolução de sistema linear.

Capítulo 9

Cifragem e Decifragem

9.1 Cifragem de texto claro

A classe `Encryptor` armazena uma cópia da chave pública, um `Sampler` e um `NttCtx`. O método `encrypt(pt)` sorteia amostras frescas a cada chamada, garantindo segurança semântica sob RLWE.

Algoritmo 12 `encrypt`: cifragem de texto claro no esquema CKKS

Require: Texto claro $m \in \mathcal{R}_q$; chave pública (p_0, p_1); (N, q, σ)
Ensure: Texto cifrado (c_0, c_1) com $c_0 + c_1 \cdot s \approx m$

$u \leftarrow \texttt{ternary_uniform}(N, q)$
$e_0 \leftarrow \texttt{gaussian}(N, q, \sigma)$
$e_1 \leftarrow \texttt{gaussian}(N, q, \sigma)$
$p_1 u \leftarrow \texttt{poly_ntt_mul}(p_1, u)$
$p_0 u \leftarrow \texttt{poly_ntt_mul}(p_0, u)$
for $i \leftarrow 0$ **até** $N - 1$ **do**
    $c_1[i] \leftarrow \texttt{mod_add}(p_1 u[i],\ e_1[i],\ q)$ ▷ $c_1 = p_1 u + e_1$
end for
for $i \leftarrow 0$ **até** $N - 1$ **do**
    $c_0[i] \leftarrow \texttt{mod_add}(\texttt{mod_add}(p_0 u[i],\ e_0[i],\ q),\ m[i],\ q)$ ▷ $c_0 = p_0 u + e_0 + m$
end for
return (c_0, c_1)

9.2 Análise do ruído de cifragem

A quantidade decifrada é computada explicitando a invariante (8.1):

$$\begin{aligned} c_0 + c_1 \cdot s &= (p_0 u + e_0 + m) + (p_1 u + e_1) \cdot s \\ &= u \cdot (p_0 + p_1 s) + e_0 + e_1 s + m \\ &= u \cdot e_{\text{chave}} + e_0 + e_1 s + m. \end{aligned} \tag{9.1}$$

O termo de erro $\varepsilon = u \cdot e_{\text{chave}} + e_0 + e_1 s$ é um polinômio com coeficientes delimitados em norma infinita por aproximadamente $2N \cdot \text{tail} \cdot \sigma \ll q/2$, garantindo que a decifragem recupere m com erro controlado. Após a decodificação por Δ, o erro por *slot* é $|\varepsilon_{\text{slot}}| \lesssim 2N \cdot \text{tail} \cdot \sigma / \Delta$, expressão que coincide com o ruído criptográfico individual e da análise de agregação do Capítulo 4 da dissertação.

9.3 Decifragem

A classe `Decryptor` armazena a chave secreta e um `NttCtx`. A decifragem é uma única multiplicação polinomial seguida de adição coeficiente a coeficiente.

Algoritmo 13 `decrypt`: decifragem de texto cifrado CKKS

Require: Texto cifrado (c_0, c_1); chave secreta s
Ensure: $m' = c_0 + c_1 \cdot s \approx m$ (texto claro ruidoso)

```
c1s ← poly_ntt_mul(c1, s)
for i ← 0 até N − 1 do
    m'[i] ← mod_add(c0[i], c1s[i], q)
end for
return m'
```

9.4 Pipeline de cifragem e decifragem

A fig. 9.1 resume o fluxo completo, incluindo a etapa de avaliação homomórfica descrita no capítulo 10.

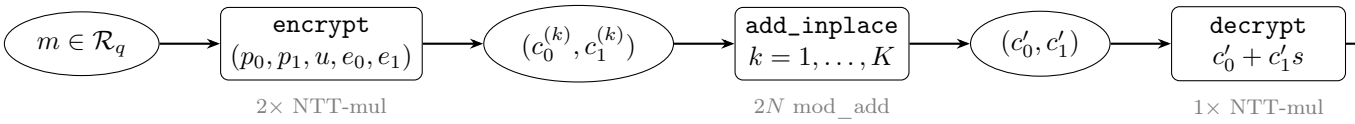


Figura 9.1: Pipeline de cifragem, avaliação homomórfica por K adições e decifragem. As operações dominantes são as multiplicações polinomiais via NTT ($O(N \log N)$ cada).

Capítulo 10

Avaliador Homomórfico Aditivo

A classe `Evaluator` é sem estado além dos parâmetros (N, q) e todos os seus métodos são seguros para uso concorrente sobre textos cifrados distintos. O método `add_inplace` realiza a adição coeficiente a coeficiente das duas componentes, conforme o algoritmo 14.

Algoritmo 14 `add_inplace`: adição homomórfica in-place

Require: Textos cifrados $(c_0^{(1)}, c_1^{(1)})$ e $(c_0^{(2)}, c_1^{(2)})$ com mesmo (N, q)
Ensure: $(c_0^{(1)}, c_1^{(1)}) \leftarrow (c_0^{(1)} + c_0^{(2)},\ c_1^{(1)} + c_1^{(2)}) \bmod q$
1: **for** $i \leftarrow 0$ **até** $N-1$ **do** ▷ Laço vetorizado: `#pragma omp simd`
2: $\quad c_0^{(1)}[i] \leftarrow \texttt{mod_add}(c_0^{(1)}[i],\ c_0^{(2)}[i],\ q)$
3: **end for**
4: **for** $i \leftarrow 0$ **até** $N-1$ **do** ▷ Laço vetorizado: `#pragma omp simd`
5: $\quad c_1^{(1)}[i] \leftarrow \texttt{mod_add}(c_1^{(1)}[i],\ c_1^{(2)}[i],\ q)$
6: **end for**

Correção. A propriedade homomórfica verifica-se diretamente:

$$\begin{aligned}(c_0^{(1)} + c_0^{(2)}) + (c_1^{(1)} + c_1^{(2)}) \cdot s &= (c_0^{(1)} + c_1^{(1)} s) + (c_0^{(2)} + c_1^{(2)} s) \\ &\approx m_1 + m_2. \end{aligned} \tag{10.1}$$

Crescimento do ruído. Como demonstrado na análise de agregação do Capítulo 4 da dissertação, após K adições de textos cifrados independentes, o ruído acumulado cresce em norma RMS como $\sqrt{K} \cdot \sigma_e$, onde σ_e é o desvio padrão do ruído individual de cifragem. O erro por *slot* após decodificação é

$$\varepsilon_K \approx \frac{\sqrt{K} \cdot \sigma_e}{\Delta}, \tag{10.2}$$

que é a mesma expressão $\sqrt{K} \cdot e/\Delta$ da equação de erro agregado da dissertação.

Capítulo 11

Codificação Canônica CKKS

O codificador converte vetores de números reais ou complexos em polinômios de $\mathcal{R}_q$ e vice-versa, por meio do mapa de imersão canônica σ [1].

11.1 Mapa de imersão canônica

Dado um polinômio $f \in \mathbb{Z}[X]/(X^N + 1)$, a imersão canônica avalia f nas N raízes primitivas $2N$-ésimas da unidade:

$$\omega_j = e^{i\pi(2j+1)/N}, \quad j = 0, 1, \ldots, N-1. \tag{11.1}$$

Como $\omega_{N-1-j} = \overline{\omega_j}$, a imagem $\sigma(f)$ exibe simetria hermitiana, de modo que apenas $N/2$ valores são independentes, definindo os $N/2$ *slots* do esquema CKKS.

11.2 Codificação (σ^{-1})

O algoritmo 15 recebe um vetor $z \in \mathbb{C}^{N/2}$, aplica a imersão inversa e quantiza os coeficientes com a escala Δ.

Algoritmo 15 `encode`: codificação canônica CKKS

Require: $z \in \mathbb{C}^{N/2}$; escala Δ; parâmetros (N, q)
Ensure: Polinômio $m \in \mathcal{R}_q$ tal que $\sigma(m/\Delta) \approx z$

$v[i] \leftarrow z[i]$ para $i = 0, \ldots, N/2 - 1$
$v[N-1-i] \leftarrow \overline{z[i]}$ para $i = 0, \ldots, N/2 - 1$ ▷ Simetria hermitiana
$v \leftarrow \texttt{FFT}(v)$
for $k \leftarrow 0$ **até** $N - 1$ **do**
    $a_k \leftarrow \mathrm{Re}(v[k] \cdot e^{-i\pi k/N})/N$ ▷ Torção e normalização
    $c \leftarrow \lfloor \Delta \cdot a_k \rceil$ ▷ Escalonamento e arredondamento
    $r \leftarrow c \bmod q$; **se** $r < 0$ **então** $r \leftarrow r + q$
    $m[k] \leftarrow r$
end for
return m

O erro de quantização por coeficiente é delimitado por $1/(2\Delta)$, produzindo erro por *slot* de magnitude $\sim N/(2\Delta)$, conforme a equação de erro de codificação $|\varepsilon_{\text{enc}}| \leq 1/(2\Delta)$ do Capítulo 4 da dissertação.

11.3 Decodificação (σ)

A decodificação inverte o processo: centraliza os coeficientes em $(-q/2, q/2]$, divide por Δ, aplica a torção e a IFFT.

Algoritmo 16 `decode`: decodificação canônica CKKS

Require: Polinômio $m \in \mathcal{R}_q$; escala Δ; parâmetros (N, q)
Ensure: $z \approx \sigma(m/\Delta) \in \mathbb{C}^{N/2}$

1: **for** $k \leftarrow 0$ **até** $N-1$ **do**
2: $\quad c \leftarrow m[k]$; **se** $c > q/2$ **então** $c \leftarrow c - q$ ▷ Centraliza em $(-q/2,\, q/2]$
3: $\quad b[k] \leftarrow (c/\Delta) \cdot e^{i\pi k/N}$ ▷ Desnormalizção e torção
4: **end for**
5: $v \leftarrow N \cdot \texttt{IFFT}(b)$ ▷ IFFT com normalização $\times N$ para desfazer o $1/N$ da FFT
6: **return** $v[0], \ldots, v[N/2-1]$

A condição $q > 2\Delta \cdot \max_j |z_j|$ deve ser satisfeita para que a centralização produza o coeficiente correto.

Capítulo 12

Multiplicação por Texto Claro (Referência)

A função `mul_plain(ct, pt)` implementa a multiplicação de um texto cifrado por um texto claro por meio do algoritmo escolar $O(N^2)$ no domínio de Montgomery. Ela serve como implementação de referência para verificação de correção e não é utilizada na produção da biblioteca, onde a multiplicação via NTT será adicionada em versão futura.

A redução negacíclica $X^N \equiv -1$ é aplicada inline: produtos de coeficientes com índice $i + j \geq N$ contribuem com sinal negativo para o coeficiente de índice $i + j - N$.

Algoritmo 17 `mul_plain`: multiplicação escolar negacíclica em domínio de Montgomery

Require: Texto cifrado (c_0, c_1); texto claro m; contexto `MontgomeryCtx`
Ensure: Texto cifrado (r_0, r_1) com $r_b = c_b \cdot m \bmod (X^N + 1, q)$ para $b \in \{0, 1\}$

Converta todos os coeficientes para forma de Montgomery: $\hat{c}_0, \hat{c}_1, \hat{m}$
for $b \in \{0, 1\}$ **do**
    for $i \leftarrow 0$ **até** $N - 1$ **do**
        for $j \leftarrow 0$ **até** $N - 1$ **do**
            $p \leftarrow$ `mont_mul`$(\hat{c}_b[i],\ \hat{m}[j])$
            if $i + j < N$ **then**
                $r_b[i + j] \leftarrow$ `mod_add`$(r_b[i + j],\ p,\ q)$
            else
                $r_b[i + j - N] \leftarrow$ `mod_sub`$(r_b[i + j - N],\ p,\ q)$ ▷ $X^N \equiv -1$: sinal negativo
            end if
        end for
    end for
    Converta r_b de volta: `from_mont` coeficiente a coeficiente
end for
return (r_0, r_1)

Capítulo 13

Serialização

O módulo de serialização implementa um formato binário *little-endian* auto-descritivo para transporte em redes FL. A unidade atômica é a representação de um polinômio `Poly`.

13.1 Formato de fio

O leiaute de bytes de um polinômio é:

$$\underbrace{N\ (\texttt{u32, 4 B})}_{\text{grau}} \parallel \underbrace{q\ (\texttt{u64, 8 B})}_{\text{módulo}} \parallel \underbrace{c_0,\ c_1,\ \ldots,\ c_{N-1}\ (N \times 8\,\text{B})}_{\text{coeficientes em } \textit{little-endian}}, \tag{13.1}$$

totalizando $12 + 8N$ bytes por polinômio. Os objetos compostos são concatenações diretas desse bloco:

- **Ciphertext**: $[c_0\ \text{Poly}]\ [c_1\ \text{Poly}] = 2(12 + 8N)$ bytes;
- **PublicKey**: $[p_0\ \text{Poly}]\ [p_1\ \text{Poly}] = 2(12 + 8N)$ bytes;
- **SecretKey**: $[s\ \text{Poly}] = 12 + 8N$ bytes.

A fig. 13.1 ilustra o leiaute de um `Ciphertext`.

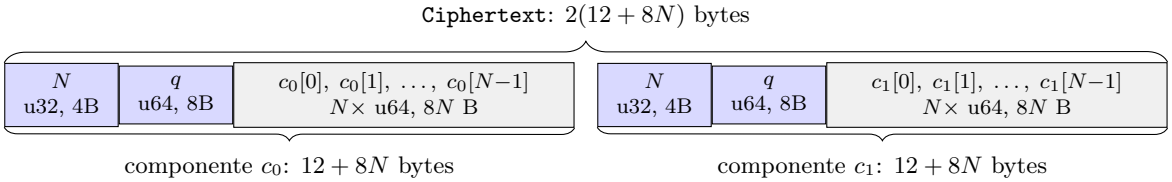


Figura 13.1: Leiaute de bytes do `Ciphertext` no formato *little-endian* auto-descritivo. Cada polinômio carrega seu próprio N e q, permitindo validação completa de integridade na desserialização.

13.2 Validação na desserialização

A função `from_bytes` verifica quatro condições antes de aceitar os dados: (i) comprimento suficiente para o cabeçalho (N, q); (ii) comprimento suficiente para os N coeficientes; (iii) concordância dos valores de N e q lidos com os esperados, prevenindo substituição silenciosa de parâmetros; e (iv) ausência de bytes residuais após o final esperado.

Capítulo 14

Compilação e Vetorização

O núcleo `ckks_core` é compilado como biblioteca estática com código independente de posição (`-fPIC`), permitindo ligação tanto a executáveis quanto ao módulo de extensão Python. A tabela 14.1 lista as diretivas de compilação e seu propósito.

Tabela 14.1: Diretivas de compilação do `ckks_core` via CMake.

Flag	**Propósito**
`-O3`	Otimizações agressivas de código (desenrolamento de laço, *inlining*)
`-march=native`	Emite instruções para a microarquitetura do *host*, habilitando AVX2
`-ffast-math`	Otimizações de ponto flutuante; utilizado principalmente pelo codificador CKKS
`-fopenmp-simd`	Ativa `#pragma omp simd` sem carregar o *runtime* de *threads* OpenMP

A diretiva `#pragma omp simd` está presente nos laços de adição de `Poly::operator+` e `Evaluator::add_inplace`. Nos laços internos da NTT, a vetorização é induzida pelos qualificadores `__restrict__` nos ponteiros de entrada e saída da borboleta, que informam ao compilador a ausência de aliasing.

Vale registrar uma divergência entre os dois caminhos de construção: o CMake inclui `-ffast-math` e `-fopenmp-simd`; a construção alternativa via `setup.py` (modo *pip*) utiliza apenas `-O3 -march=native -Wall`. Resultados numéricos do codificador podem diferir levemente entre os dois modos quando operando próximo dos limites de precisão de ponto flutuante, pois `-ffast-math` permite reordenações que alteram o arredondamento.

Capítulo 15

Conclusão e Trabalhos Futuros

Este documento apresentou a implementação de baixo nível do *backend* C++ da biblioteca CKKS customizada, organizada em torno de três pilares técnicos: (i) redução de Montgomery para eliminação de divisões na aritmética modular, com custo de construção $O(1)$ amortizado sobre toda a execução; (ii) NTT negacíclica para multiplicação polinomial em $O(N \log N)$, com tabelas de twiddle pré-computadas em forma de Montgomery e em ordem bit-reversa para acesso linear à linha de *cache*; e (iii) aritmética modular sem desvios condicionais para vetorização automática via AVX2.

As escolhas de projeto, em particular o uso de um único primo NTT em lugar de uma cadeia RNS completa e a restrição ao regime somente-aditivo, mantiveram a complexidade de implementação baixa sem comprometer a correção para o caso de uso alvo: agregação federada de gradientes por adição homomórfica.

Os trabalhos futuros imediatos incluem:

1. **NTT multithread:** paralelização dos laços de borboleta com `-fopenmp` para aproveitar múltiplos núcleos em grades $N \geq 8192$;
2. **Multiplicação escalar via NTT:** substituição de `mul_plain` por `poly_ntt_mul`, reduzindo a complexidade de $O(N^2)$ para $O(N \log N)$;
3. **Cadeia RNS:** extensão para múltiplos primos $q_1, \ldots, q_L$ com implementação de *rescaling*, habilitando profundidade multiplicativa para circuitos de FL mais complexos.

Referências Bibliográficas

[1] J. H. Cheon, A. Kim, M. Kim, e Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," em *Advances in Cryptology – ASIACRYPT 2017*, Lecture Notes in Computer Science, vol. 10624, Springer, 2017, pp. 409–437.

[2] P. L. Montgomery, "Modular multiplication without trial division," *Mathematics of Computation*, vol. 44, n. 170, pp. 519–521, abr. 1985.

[3] J. W. Cooley e J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, n. 90, pp. 297–301, 1965.

[4] P. Longa e M. Naehrig, "Speeding up the number theoretic transform for faster ideal lattice-based cryptography," em *Cryptology and Network Security – CANS 2016*, Lecture Notes in Computer Science, vol. 10052, Springer, 2016, pp. 124–139.

[5] V. Lyubashevsky, C. Peikert, e O. Regev, "On ideal lattices and learning with errors over rings," em *Advances in Cryptology – EUROCRYPT 2010*, Lecture Notes in Computer Science, vol. 6110, Springer, 2010, pp. 1–23.

[6] Microsoft Research, *SEAL: Simple Encrypted Arithmetic Library – Manual*, versão 4.1, 2023. Disponível em: https://github.com/microsoft/SEAL

[7] C. Pomerance, J. L. Selfridge, e S. S. Wagstaff Jr., "The pseudoprimes to $25 \cdot 10^9$," *Mathematics of Computation*, vol. 35, n. 151, pp. 1003–1026, 1980.

[8] M. Matsumoto e T. Nishimura, "Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator," *ACM Transactions on Modeling and Computer Simulation*, vol. 8, n. 1, pp. 3–30, jan. 1998.

[9] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3.ª ed., Addison-Wesley, 1997.